LangChain: Evaluation

Outline:

- Example generation
- Manual evaluation (and debuging)
- LLM-assisted evaluation

In [1]:
```
import os

from dotenv import load_dotenv, find_dotenv
_ = load_dotenv(find_dotenv()) # read local .env file
```

Create our QandA application

In [2]:
```
from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI
from langchain.document_loaders import CSVLoader
from langchain.indexes import VectorstoreIndexCreator
from langchain.vectorstores import DocArrayInMemorySearch
```

In [3]:
```
file = 'OutdoorClothingCatalog_1000.csv'
loader = CSVLoader(file_path=file)
data = loader.load()
```

In [ ]:
```
index = VectorstoreIndexCreator(
    vectorstore_cls=DocArrayInMemorySearch
).from_loaders([loader])
```

In [ ]:
```
llm = ChatOpenAI(temperature = 0.0)
qa = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=index.vectorstore.as_retriever(),
    verbose=True,
    chain_type_kwargs = {
        "document_separator": "<<<<>>>>>"
    }
)
```

Coming up with test datapoints

In [ ]:
```
data[10]
```

In [ ]:
```
data[11]
```

Hard-coded examples

In [6]:
```
examples = [
    {
        "query": "Do the Cozy Comfort Pullover Set\
        have side pockets?",
        "answer": "Yes"
    },
    {
        "query": "What collection is the Ultra-Lofty \
        850 Stretch Down Hooded Jacket from?",
        "answer": "The DownTek collection"
    }
]
```

LLM-Generated examples

In [7]:
```
from langchain.evaluation.qa import QAGenerateChain
```

In [8]:
```
example_gen_chain = QAGenerateChain.from_llm(ChatOpenAI())
```

In [ ]:
```
new_examples = example_gen_chain.apply_and_parse(
    [{"doc": t} for t in data[:5]]
)
```

In [ ]:
```
new_examples[0]
```

In [ ]:
```
data[0]
```

Combine examples

In [ ]:
```
examples += new_examples
```

In [ ]:
```
qa.run(examples[0]["query"])
```

Manual Evaluation

In [10]:
```
import langchain
langchain.debug = True
```

In [ ]:
```
qa.run(examples[0]["query"])
```

In [ ]:
```
# Turn off the debug mode
langchain.debug = False
```

LLM assisted evaluation

In [ ]:
```
predictions = qa.apply(examples)
```

In [11]:
```
from langchain.evaluation.qa import QAEvalChain
```

In [12]:
```
llm = ChatOpenAI(temperature=0)
eval_chain = QAEvalChain.from_llm(llm)
```

In [ ]:
```
for i, eg in enumerate(examples):
    print(f"Example {i}:")
    print("Question: " + predictions[i]['query'])
    print("Real Answer: " + predictions[i]['answer'])
    print("Predicted Answer: " + predictions[i]['result'])
    print("Predicted Grade: " + graded_outputs[i]['text'])
    print()
```